

Final Assignment: Schedule Search

SANFELIU Quentin

IPSA

April 2026

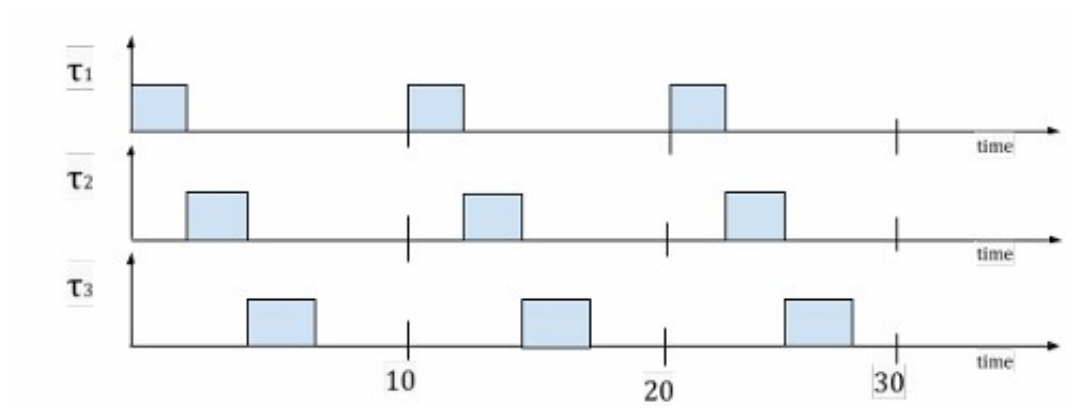


Figure 1: Illustration image: Scheduling from Mr. Singh's course

2025-2026

Table of Contents

1	Measuring the WCET of Task τ_1	3
1.1	Measurement approach	3
1.2	Results	3
2	Task Set and Schedulability Analysis	4
2.1	Task set overview	4
2.2	Assumptions	4
2.3	Necessary condition: total utilization	4
2.4	Hyperperiod	5
3	Part 1: Schedule with No Missed Deadlines	6
3.1	Manual scheduling analysis	6
3.2	Baseline: Non-Preemptive EDF	6
3.3	The case of the Shortest Job First algorithm (SJF)	7
3.4	Optimized algorithm: SJF with deadline safety check	7
4	Part 2: τ_5 Allowed to Miss Its Deadline	9
4.1	Strategy	9
4.2	Full schedule	9
4.3	Why deprioritizing τ_5 gives a worse result	9
5	Algorithm Comparison	11
6	Algorithmic Complexity	12
7	Conclusion	13

1 Measuring the WCET of Task τ_1

In this first section, we compute the WCET (Worst-Case Execution Time), which represents the longest possible execution time of a task. This value is used to dimension task execution budgets in the schedule and ensure that no deadline is ever missed.

Task τ_1 computes the product of two random integers drawn uniformly from the interval $[1\,500\,000; 2\,000\,000]$. The goal of this section is to determine its Worst-Case Execution Time (WCET), denoted C_1 , which will be used as the execution budget throughout the scheduling analysis.

1.1 Measurement approach

Our strategy is straightforward: the function `multipli_rand()` is executed 10 000 times in a tight loop, and each run is timed using `clock_gettime(CLOCK_MONOTONIC)`, imported via a dedicated module. This system clock is monotonic, meaning it never goes backwards, and offers nanosecond-level resolution. It is the standard choice for performance measurements on Linux, widely cited for this type of application.

One important change was made to the code during testing: all `printf()` calls used to track computation progress were removed. These calls were artificially inflating the task's execution time and producing unrepresentative measurements. Leaving the prints in would have completely skewed the results.

After 10 000 runs, the five key statistics were extracted. The maximum is retained as the WCET, since by definition it is the value that can never be exceeded. A safety margin of approximately 1000 was also applied to the final WCET value: this ensures system safety while keeping the value very small relative to the integer microsecond timescale of the schedule. Indeed, the smallest integer unit of 1 is already well above the raw WCET, providing a comfortable margin.

1.2 Results

Statistic	Value	Interpretation
Minimum	0 ns	Hardware resolution limit
Q1 (25 %)	91 ns	
Q2 (médiane)	92 ns	Typical execution time
Q3 (75 %)	92 ns	
Maximum = WCET (C_1)	76 542 ns	Retained value

Table 1: Statistiques d'exécution de τ_1 sur 10 000 runs

Note

The minimum value of 0 may seem surprising, but it is not a code error. This can be explicated by the fact that when the processor executes the multiplication very quickly, both calls to `clock_gettime` can land on the same hardware clock tick, giving a reported delta of 0. So this is look like a hardware resolution limit and not a code error.

The empirical WCET is therefore $C_1 = 76\,542$ ns. For the rest of the analysis, we work in **ms** by setting $C_1 = 1$ unit. What matters is the ratio between execution times and periods, not their absolute nanosecond values.

2 Task Set and Schedulability Analysis

Before writing the code, we need to verify that the tasks are actually schedulable, meaning that a valid schedule exists. This section presents the tasks, the assumptions we make, and the schedulability checks.

2.1 Task set overview

The system consists of 7 periodic tasks with implicit deadlines ($D_i = T_i$), all released simultaneously at $t = 0$. With $C_1 = 1$ ms:

Task	C_i (WCET)	T_i (Period)	D_i (Deadline)	$U_i = C_i/T_i$	Jobs in H
τ_1	1	10	10	0.1000	8
τ_2	3	10	10	0.3000	8
τ_3	2	20	20	0.1000	4
τ_4	2	20	20	0.1000	4
τ_5	2	40	40	0.0500	2
τ_6	2	40	40	0.0500	2
τ_7	3	80	80	0.0375	1
Total	—	—	—	0.7375	29

Table 2: Task set characteristics

2.2 Assumptions

Here are the assumptions we made for this work. It is important to state them explicitly, as all results below depend on them:

- **Single processor:** all tasks share one CPU, and only one job runs at a time.
- **Non-preemptive:** once a job has started, it runs to completion without interruption. This is more restrictive than preemptive scheduling and requires more careful deadline analysis.
- **Implicit deadlines:** $D_i = T_i$ for each task, so every job must finish before the next instance of the same task is released.
- **Synchronous release:** all tasks release their first job at $t = 0$. This is the most pessimistic phasing in terms of utilization, and it is the standard convention for computing the hyperperiod.
- **Independence:** no precedence constraints, no shared resources, no blocking.
- **Deterministic execution times:** we use WCET values, so each job is assumed to take exactly C_i time units.

2.3 Necessary condition: total utilization

The first check is processor utilization. If the sum of all C_i/T_i exceeds 1.0, the processor simply does not have enough time to run all the tasks. No algorithm can fix that.

$$U = \sum_{i=1}^7 \frac{C_i}{T_i} = \frac{1}{10} + \frac{3}{10} + \frac{2}{20} + \frac{2}{20} + \frac{2}{40} + \frac{2}{40} + \frac{3}{80} = 0.7375$$

$U = 0.7375 < 1.0$: the necessary condition is satisfied. The processor is busy 73.75% of the time, leaving 26.25% idle. Over a hyperperiod of 80 units, this means 59 units of computation and 21 idle units, a constant of the task set that is independent of the scheduling algorithm.

2.4 Hyperperiod

The hyperperiod H is the Least Common Multiple (LCM) of all task periods. It is the smallest interval after which the execution pattern repeats identically. Analyzing one hyperperiod is therefore sufficient to characterize the system's behavior over an infinite duration.

$$H = \text{LCM}(10, 10, 20, 20, 40, 40, 80) = 80 \text{ time units}$$

The total number of jobs in this hyperperiod is:

$$J = \frac{80}{10} + \frac{80}{10} + \frac{80}{20} + \frac{80}{20} + \frac{80}{40} + \frac{80}{40} + \frac{80}{80} = 8 + 8 + 4 + 4 + 2 + 2 + 1 = 29 \text{ jobs}$$

These 29 jobs represent exactly 59 units of CPU load, consistent with $U \times H = 0.7375 \times 80 = 59$.

3 Part 1: Schedule with No Missed Deadlines

Our goal is to find a non-preemptive schedule that respects all 29 deadlines while minimizing total waiting time between tasks. We explore three approaches in order of increasing complexity.

3.1 Manual scheduling analysis

To approach this problem, we can first think analytically and try to organize the tasks as efficiently as possible over one hyperperiod of 80 ms. The key idea is to first respect each task's deadline, then order tasks by increasing execution time. This avoids blocking short tasks behind long ones, which directly reduces waiting time.

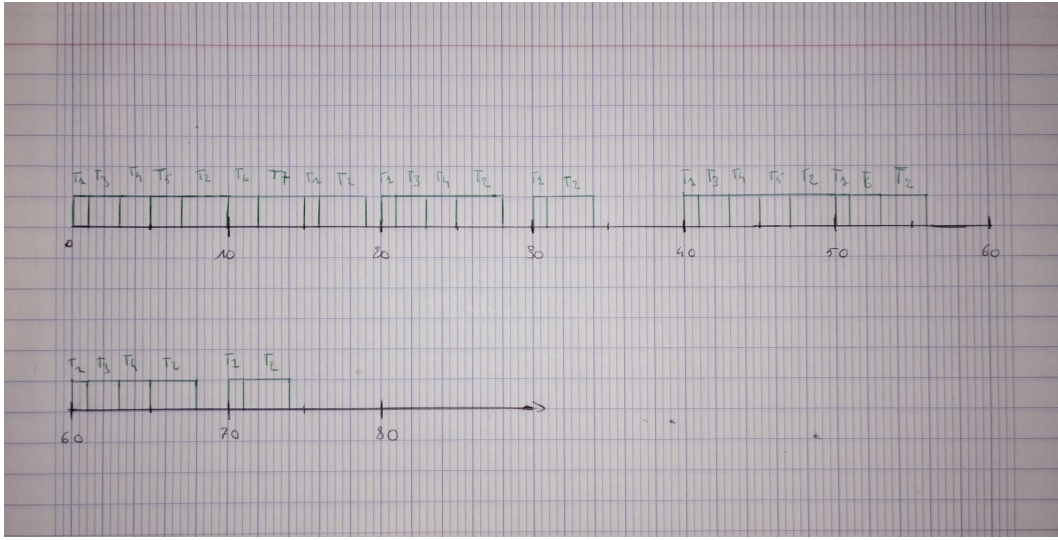


Figure 2: Analytical scheduling by hand

The problem with implementing such a schedule manually is that it can become complex in the general case, combining several competing priorities. A task-by-task approach is always possible but quickly becomes tedious and is not viable in the general case.

3.2 Baseline: Non-Preemptive EDF

We therefore look for algorithms that better match our needs, namely algorithms that are more general and simpler to implement. The first one that naturally comes to mind in this context is Earliest Deadline First (EDF).

EDF (*Earliest Deadline First*) is a natural starting point when deadlines must not be missed. At each CPU release, EDF selects the available job with the closest absolute deadline. This ensures that the most urgent tasks are always processed first, which provides a safety guarantee.

Here is the complete schedule over the hyperperiod, with each job's response time ($R_i = \text{finish} - \text{release}$) verified against its deadline:

Job	Release	Deadline	Start	Fin	Waiting	Status
$\tau_1[0]$	0	10	0	1	0	ok
$\tau_2[0]$	0	10	1	4	1	ok
$\tau_3[0]$	0	20	4	6	4	ok
$\tau_4[0]$	0	20	6	8	6	ok
$\tau_5[0]$	0	40	8	10	8	ok
$\tau_1[1]$	10	20	10	11	0	ok
$\tau_2[1]$	10	20	11	14	1	ok
$\tau_6[0]$	0	40	14	16	14	ok
$\tau_7[0]$	0	80	16	19	16	ok
$\tau_1[2]$	20	30	20	21	0	ok
$\tau_2[2]$	20	30	21	24	1	ok
$\tau_3[1]$	20	40	24	26	4	ok
$\tau_4[1]$	20	40	26	28	6	ok
$\tau_1[3]$	30	40	30	31	0	ok
$\tau_2[3]$	30	40	31	34	1	ok
$\tau_1[4]$	40	50	40	41	0	ok
$\tau_2[4]$	40	50	41	44	1	ok
$\tau_3[2]$	40	60	44	46	4	ok
$\tau_4[2]$	40	60	46	48	6	ok
$\tau_5[1]$	40	80	48	50	8	ok
$\tau_1[5]$	50	60	50	51	0	ok
$\tau_2[5]$	50	60	51	54	1	ok
$\tau_6[1]$	40	80	54	56	14	ok
$\tau_1[6]$	60	70	60	61	0	ok
$\tau_2[6]$	60	70	61	64	1	ok
$\tau_3[3]$	60	80	64	66	4	ok
$\tau_4[3]$	60	80	66	68	6	ok
$\tau_1[7]$	70	80	70	71	0	ok
$\tau_2[7]$	70	80	71	74	1	ok
Total					108	0 miss

Table 3: Non-preemptive EDF schedule over $H = 80$

EDF works correctly, with no missed deadlines. But with a total waiting time of 108, one might wonder whether we can do better. EDF does not care about waiting time; it only seeks to avoid deadline violations. Shorter jobs could sometimes run before longer ones without putting any deadline at risk. There seems to be room for improvement.

3.3 The case of the Shortest Job First algorithm (SJF)

SJF (*Shortest Job First*) is the classical algorithm for minimizing average waiting time: at each CPU release, the shortest available job is executed first. Applied naively here, SJF achieves a waiting time of 96. However, it does so with 2 missed deadlines, which breaks one of our fundamental rules and makes it invalid as-is.

The reason is visible from $t = 0$. All 7 first-instance jobs are available simultaneously. SJF ranks them by duration: τ_1 ($C = 1$), then $\tau_3, \tau_4, \tau_5, \tau_6$ ($C = 2$ each), then τ_2 and τ_7 ($C = 3$). If all five short jobs run before τ_2 , they consume $1 + 2 + 2 + 2 + 2 = 9$ units. τ_2 can only start at $t = 9$, finishes at $t = 12$, and misses its deadline of 10. Pure SJF is therefore invalid for this task set as it stands.

3.4 Optimized algorithm: SJF with deadline safety check

The optimal solution appears to be a combination of both approaches. We keep SJF as the primary strategy, but add a feasibility check before each decision: if running the shortest job risks causing another job to miss its deadline, we fall back to EDF for that single choice. The algorithm, at each CPU release, works as follows:

1. **SJF step:** identify the shortest available job (SJF candidate).

2. **Feasibility check:** for each other available job j , verify: if we run the SJF candidate now, will j miss its deadline? The danger condition is:

$$t_{\text{actuel}} + C_{\text{SJF}} + C_j > D_j$$

3. **Decision:** if a job j is endangered $\rightarrow$ run the most urgent one instead (EDF fallback). Otherwise $\rightarrow$ SJF.

In practice, this check only triggers when τ_2 ($C = 3$, $T = 10$) risks being pushed too far due to its tight deadline. In all other cases, SJF applies freely. We thus get the best of both worlds: minimum waiting time and guaranteed deadlines.

Here is the complete schedule produced by the algorithm:

Job	Release	Deadline	Start	Fin	Waiting	Status
$\tau_1[0]$	0	10	0	1	0	ok
$\tau_3[0]$	0	20	1	3	1	ok
$\tau_4[0]$	0	20	3	5	3	ok
$\tau_5[0]$	0	40	5	7	5	ok
$\tau_2[0]$	0	10	7	10	7	ok
$\tau_1[1]$	10	20	10	11	0	ok
$\tau_6[0]$	0	40	11	13	11	ok
$\tau_2[1]$	10	20	13	16	3	ok
$\tau_7[0]$	0	80	16	19	16	ok
$\tau_1[2]$	20	30	20	21	0	ok
$\tau_3[1]$	20	40	21	23	1	ok
$\tau_4[1]$	20	40	23	25	3	ok
$\tau_2[2]$	20	30	25	28	5	ok
$\tau_1[3]$	30	40	30	31	0	ok
$\tau_2[3]$	30	40	31	34	1	ok
$\tau_1[4]$	40	50	40	41	0	ok
$\tau_3[2]$	40	60	41	43	1	ok
$\tau_4[2]$	40	60	43	45	3	ok
$\tau_5[1]$	40	80	45	47	5	ok
$\tau_2[4]$	40	50	47	50	7	ok
$\tau_1[5]$	50	60	50	51	0	ok
$\tau_6[1]$	40	80	51	53	11	ok
$\tau_2[5]$	50	60	53	56	3	ok
$\tau_1[6]$	60	70	60	61	0	ok
$\tau_3[3]$	60	80	61	63	1	ok
$\tau_4[3]$	60	80	63	65	3	ok
$\tau_2[6]$	60	70	65	68	5	ok
$\tau_1[7]$	70	80	70	71	0	ok
$\tau_2[7]$	70	80	71	74	1	ok
Total					96	0 miss

Table 4: SJF + EDF schedule over $H = 80$ (Part 1)

Result

Total waiting time drops from 108 (EDF) to **96** (SJF+EDF), a reduction of 11.1%, with zero missed deadlines. CPU busy time remains identical: $59/80 = 73.75\%$. The 21 idle units are a mathematical constant of the task set and no algorithm can change that.

4 Part 2: τ_5 Allowed to Miss Its Deadline

In the second part of the assignment, τ_5 is allowed to miss its deadline, with the expectation that this extra freedom might further reduce total waiting time. We examine here whether that is actually the case.

4.1 Strategy

The natural interpretation is to treat τ_5 as a deprioritized task: it only competes for the CPU when no other job is ready or at risk. Concretely, τ_5 is excluded from the SJF+EDF selection process. The 6 other tasks are scheduled as before, and τ_5 only runs in slots considered idle.

4.2 Full schedule

Job	Release	Deadline	Start	Fin	Waiting	Status
$\tau_1[0]$	0	10	0	1	0	ok
$\tau_3[0]$	0	20	1	3	1	ok
$\tau_4[0]$	0	20	3	5	3	ok
$\tau_6[0]$	0	40	5	7	5	ok
$\tau_2[0]$	0	10	7	10	7	ok
$\tau_1[1]$	10	20	10	11	0	ok
$\tau_2[1]$	10	20	11	14	1	ok
$\tau_7[0]$	0	80	14	17	14	ok
$\tau_5[0]$	0	40	17	19	17	ok
$\tau_1[2]$	20	30	20	21	0	ok
$\tau_3[1]$	20	40	21	23	1	ok
$\tau_4[1]$	20	40	23	25	3	ok
$\tau_2[2]$	20	30	25	28	5	ok
$\tau_1[3]$	30	40	30	31	0	ok
$\tau_2[3]$	30	40	31	34	1	ok
$\tau_1[4]$	40	50	40	41	0	ok
$\tau_3[2]$	40	60	41	43	1	ok
$\tau_4[2]$	40	60	43	45	3	ok
$\tau_6[1]$	40	80	45	47	5	ok
$\tau_2[4]$	40	50	47	50	7	ok
$\tau_1[5]$	50	60	50	51	0	ok
$\tau_2[5]$	50	60	51	54	1	ok
$\tau_5[1]$	40	80	54	56	14	ok
$\tau_1[6]$	60	70	60	61	0	ok
$\tau_3[3]$	60	80	61	63	1	ok
$\tau_4[3]$	60	80	63	65	3	ok
$\tau_2[6]$	60	70	65	68	5	ok
$\tau_1[7]$	70	80	70	71	0	ok
$\tau_2[7]$	70	80	71	74	1	ok
Total					99	0 miss

Table 5: SJF+EDF schedule with τ_5 deprioritized (Part 2), with τ_5 highlighted in yellow

4.3 Why deprioritizing τ_5 gives a worse result

The result we obtain is counterintuitive: we were given more freedom, yet the outcome is worse (99 vs 96). The explanation lies in how τ_5 fits into the schedule.

In Part 1, τ_5 has $C = 2$ and is available from $t = 0$. SJF naturally inserts it at $t = 5 \rightarrow 7$, right after τ_1 , τ_3 and τ_4 (all shorter or equal in duration). It is a logical slot based on job lengths, and τ_5 contributes with a waiting time of only 5 units.

In Part 2, τ_5 is excluded. τ_6 (also $C = 2$, available at $t = 0$) takes its slot at $t = 5 \rightarrow 7$. τ_5 is pushed back and can only run at $t = 17 \rightarrow 19$, a natural but much later slot. Its waiting time jumps from 5 to 17 for $\tau_5[0]$, and from 5 to 14 for $\tau_5[1]$. The other tasks do not benefit enough from τ_5 's absence to compensate for this loss.

The exact difference, job by job, between the two parts is as follows:

Job	Waiting Part 1	Waiting Part 2	Difference
$\tau_6[0]$	11	5	-6
$\tau_7[0]$	16	14	-2
$\tau_2[1]$	3	1	-2
$\tau_5[0]$	5	17	+12
$\tau_5[1]$	5	14	+9
$\tau_2[5]$	3	1	-2
Net	96	99	+3

Table 6: Waiting time differences between Part 1 and Part 2

It is clear that the gains on $\tau_6[0]$, $\tau_6[1]$, $\tau_7[0]$, $\tau_2[1]$ and $\tau_2[5]$ ($-6 - 6 - 2 - 2 - 2 = -18$) do not compensate the losses on $\tau_5[0]$ and $\tau_5[1]$ ($+12 + 9 = +21$). Net result: +3 units, which explains the gap $96 \rightarrow 99$.

It therefore appears that 96 is the absolute minimum and that altering the previous algorithm's schedule is counterproductive when trying to minimize waiting time. This means τ_5 was already placed in the best possible slots. There does not seem to be a better configuration by deprioritizing τ_5 .

5 Algorithm Comparison

Algorithm	Waiting totale	Missed Deadlines	Note
EDF non-préemptif	108	0	Baseline
SJF pur	96	2	Invalid
SJF + EDF (Part 1)	96	0	Optimal
SJF + EDF, τ_5 deprioritized (P2)	99	0	Worse than P1

Table 7: Algorithm comparison summary

A few points worth noting explicitly:

- **Idle time is identical for all algorithms** ($21/80 = 26.25\%$). This is a constant of the task set: total load is 59 units, so there will always be 21 idle units regardless of the scheduling policy.
- **SJF+EDF achieves the same waiting time as pure SJF (96)**, but without the deadline violations. The feasibility check prevents those violations at no extra cost in waiting time.
- **Allowing τ_5 to miss its deadline brings no benefit**. τ_5 fits naturally into the optimal schedule and did not need any special freedom.

6 Algorithmic Complexity

Let n be the number of tasks and H the hyperperiod. The total number of jobs over the hyperperiod is:

$$J = \sum_{i=1}^n \frac{H}{T_i}$$

For our task set, $J = 29$. The algorithm works as follows:

- The main loop runs J times, once per job to schedule.
- At each iteration, all J jobs are scanned twice: once to find the SJF candidate, once for the feasibility check.

The total number of operations is therefore proportional to $J \times J = J^2$. Since J is a linear function of n (for fixed H and periods), the complexity as a function of the number of tasks is:

Time complexity = $O(n^2)$

For our case, this represents at most $29^2 = 841$ comparisons, which is negligible on any embedded platform. An implementation using a priority queue would reduce complexity to $O(J \log J)$, but there is no practical motivation here: the hyperperiod is short, J is small, and the simple scan is easier to verify for correctness.

Space complexity is $O(J) = O(n)$ to store the job list, with one struct per job statically allocated.

7 Conclusion

This report has addressed the complete scheduling problem for a 7-task real-time system under non-preemptive single-processor constraints.

The task set is schedulable: total utilization $U = 0.7375 < 1.0$ satisfies the necessary condition, and the simulation over the full hyperperiod confirms that all 29 jobs complete before their respective deadlines.

For Part 1, the SJF algorithm augmented with an EDF feasibility check achieves a total waiting time of 96 units with zero missed deadlines. The improvement over pure EDF (108 to 96, -11.1%) comes from reordering jobs within each scheduling window, a simple approach that turns out to be surprisingly effective.

For Part 2, the freedom granted to τ_5 to miss its deadline yields no benefit. Deprioritizing τ_5 pushes it into a later slot, increases its waiting time from 5 to 17, and raises the total to 99, which is worse than Part 1. τ_5 was not a problem in the optimal schedule and did not need any special treatment.

The algorithm runs in $O(n^2)$ and processes all 29 jobs in under 1000 operations, well within the constraints of any real-time embedded platform.